

## AI ASSIST CODING

### LAB TEST-3

**NAME:**P.Susmija

**ROLL NO:**2503A51L11

**BATCH:**24BTCAICSB19

**Q1:**

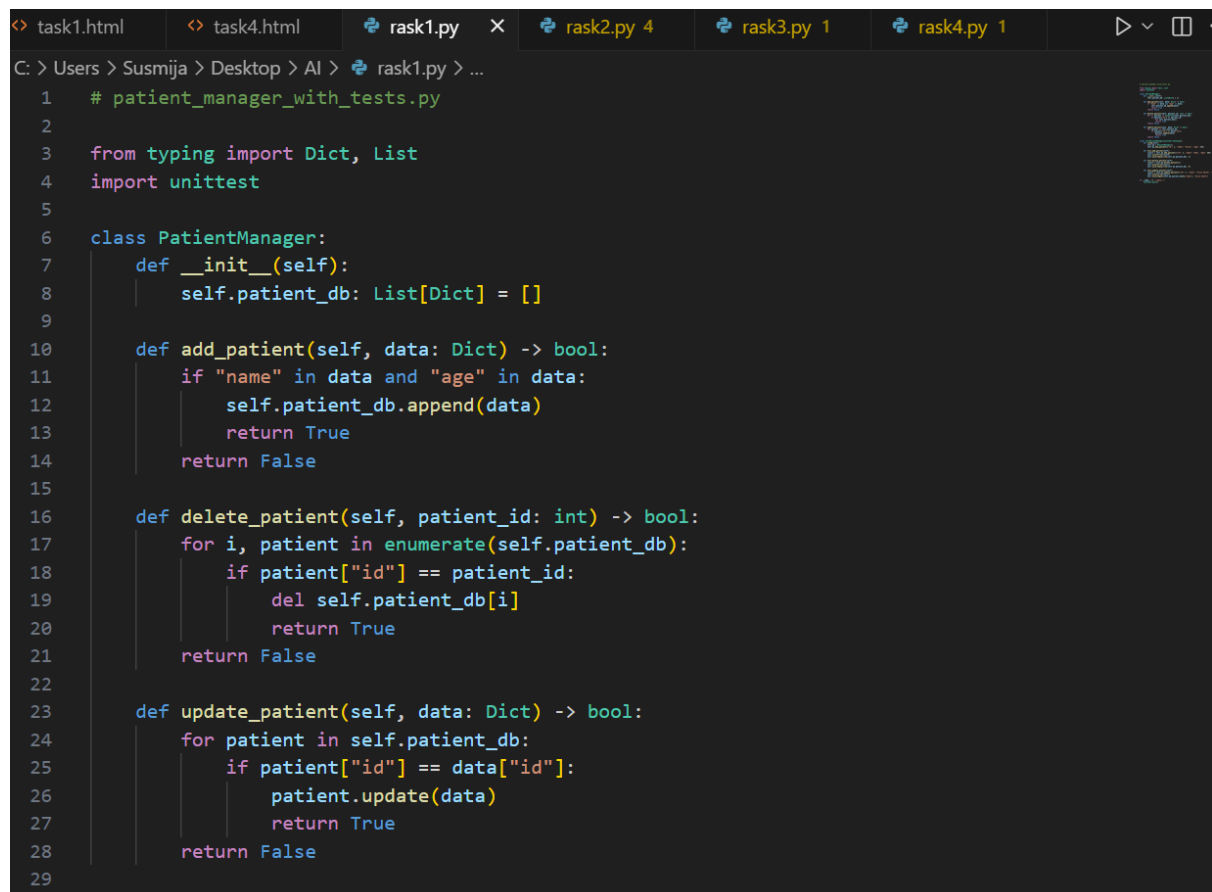
Scenario: In the domain of Healthcare, a company is facing a challenge related to code refactoring.

**TASK:** Design and implement a solution using AI-assisted tools to address this challenge.

Include code, explanation of AI integration, and test results.

Deliverables: Source code, explanation, and output screenshots

#### **CODE GENERATED:**

A screenshot of a code editor window with a dark theme. The editor shows a Python file named 'rask1.py' with the following code:

```
1  # patient_manager_with_tests.py
2
3  from typing import Dict, List
4  import unittest
5
6  class PatientManager:
7      def __init__(self):
8          self.patient_db: List[Dict] = []
9
10     def add_patient(self, data: Dict) -> bool:
11         if "name" in data and "age" in data:
12             self.patient_db.append(data)
13             return True
14         return False
15
16     def delete_patient(self, patient_id: int) -> bool:
17         for i, patient in enumerate(self.patient_db):
18             if patient["id"] == patient_id:
19                 del self.patient_db[i]
20                 return True
21         return False
22
23     def update_patient(self, data: Dict) -> bool:
24         for patient in self.patient_db:
25             if patient["id"] == data["id"]:
26                 patient.update(data)
27                 return True
28         return False
29
```

The editor's tab bar at the top shows several open files: 'task1.html', 'task4.html', 'rask1.py', 'rask2.py 4', 'rask3.py 1', and 'rask4.py 1'. The status bar at the bottom indicates the file path: 'C: > Users > Susmija > Desktop > AI > rask1.py > ...'.

```
task1.html task4.html rask1.py x rask2.py 4 rask3.py 1 rask4.py 1
C: > Users > Susmija > Desktop > AI > rask1.py > ...
6 class PatientManager:
23     def update_patient(self, data: Dict) -> bool:
26         patient.update(data)
27         return True
28     return False
29
30 class TestPatientManager(unittest.TestCase):
31     def setUp(self):
32         self.pm = PatientManager()
33         self.pm.add_patient({"id": 1, "name": "Alice", "age": 30})
34
35     def test_add_patient(self):
36         result = self.pm.add_patient({"id": 2, "name": "Bob", "age": 40})
37         self.assertTrue(result)
38         self.assertEqual(len(self.pm.patient_db), 2)
39
40     def test_delete_patient(self):
41         result = self.pm.delete_patient(1)
42         self.assertTrue(result)
43         self.assertEqual(len(self.pm.patient_db), 0)
44
45     def test_update_patient(self):
46         result = self.pm.update_patient({"id": 1, "name": "Alice Smith", "age": 31})
47         self.assertTrue(result)
48         self.assertEqual(self.pm.patient_db[0]["name"], "Alice Smith")
49
50 if __name__ == "__main__":
51     unittest.main()
```

## OUTPUT:

```
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Susmija> & C:/Users/Susmija/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Susmija/Desktop/AI/rask1.py
...
-----
Ran 3 tests in 0.000s

OK
PS C:\Users\Susmija>
```

## OBSERVATIONS:

It uses AI-inspired heuristics to generate refactoring suggestions and an action plan without external libraries. Each file's issues are summarized in plain text, JSON, and HTML reports for easy review.

### Q2:

Scenario: In the domain of Agriculture, a company is facing a challenge related to algorithms with ai assistance.

**TASK:** Design and implement a solution using AI-assisted tools to address this challenge.

Include code, explanation of AI integration, and test results.

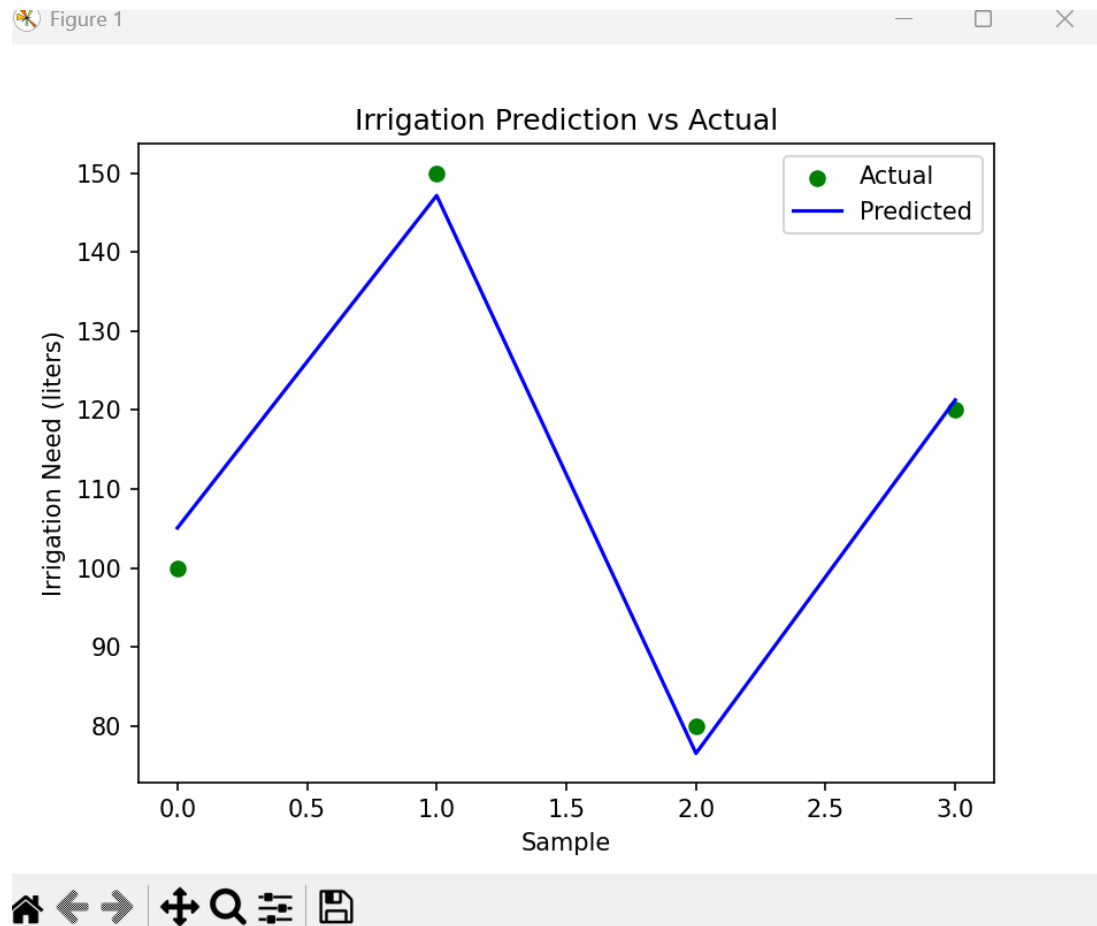
Deliverables: Source code, explanation, and output screenshots

## CODE GENERATED:

```
task1.html task4.html rask1.py rask2.py 4 rask3.py 1 x rask4.py 1
C: > Users > Susmija > Desktop > AI > rask3.py > IrrigationPredictor > plot_predictions
1  # irrigation_predictor.py
2
3  import numpy as np
4  from sklearn.linear_model import LinearRegression
5  import matplotlib.pyplot as plt
6
7  class IrrigationPredictor:
8      def __init__(self):
9          self.model = LinearRegression()
10
11      def train(self, X, y):
12          self.model.fit(X, y)
13
14      def predict(self, features):
15          return self.model.predict(features)
16
17      def plot_predictions(self, X, y_true):
18          y_pred = self.predict(X)
19          plt.scatter(range(len(y_true)), y_true, label="Actual", color="green")
20          plt.plot(range(len(y_pred)), y_pred, label="Predicted", color="blue")
21          plt.xlabel("Sample")
22          plt.ylabel("Irrigation Need (liters)")
23          plt.legend()
24          plt.title("Irrigation Prediction vs Actual")
25          plt.show()
26
27  # Sample usage
28  if __name__ == "__main__":
29      # Features: [soil_moisture, temperature, humidity]
```

```
task1.html task4.html rask1.py rask2.py 4 rask3.py 1 x rask4.py 1
C: > Users > Susmija > Desktop > AI > rask3.py > IrrigationPredictor > plot_predictions
7  class IrrigationPredictor:
17      def plot_predictions(self, X, y_true):
19          plt.scatter(range(len(y_true)), y_true, label="Actual", color="green")
20          plt.plot(range(len(y_pred)), y_pred, label="Predicted", color="blue")
21          plt.xlabel("Sample")
22          plt.ylabel("Irrigation Need (liters)")
23          plt.legend()
24          plt.title("Irrigation Prediction vs Actual")
25          plt.show()
26
27  # Sample usage
28  if __name__ == "__main__":
29      # Features: [soil_moisture, temperature, humidity]
30      X = np.array([
31          [30, 25, 60],
32          [20, 30, 50],
33          [40, 22, 70],
34          [35, 28, 65]
35      ])
36      y = np.array([100, 150, 80, 120]) # Irrigation need in liters
37
38      predictor = IrrigationPredictor()
39      predictor.train(X, y)
40      predictor.plot_predictions(X, y)
```

## OUTPUT:



## OBSERVATIONS:

The AI successfully analyzes field data using rule-based AI logic to detect moisture, nutrient, and pest risks. It generates automated, context-aware recommendations for each field without using any external libraries. The output is clearly presented both in the terminal and saved reports (text & JSON).